

Near-Memory LLM Inference Processor based on 3D DRAM-to-logic Hybrid Bonding

Sanghyeok Han*, Byungkuk Yoon*, Gyeonghwan Park*, Choungki Song[†], Dongkyun Kim[†] and Jae-Joon Kim*
Seoul National University*, SK Hynix, DRAM Development[†]
{hansh778, bkyoon, gyeonghwan.park, kimjaejoon}@snu.ac.kr, {choungki.song, dongkyun.kim}@sk.com

Abstract—Large language model (LLM) inference poses dual challenges, demanding substantial memory bandwidth and computing resources. Recent advancements in near-memory accelerators leveraging 3D DRAM-to-logic hybrid-bonding (HB) interconnects have gained attention due to their highly parallel data transfer capabilities. We address limitations in previous HB-DRAM accelerators, such as those stemming from distributed controller designs, by introducing an architecture with a centralized controller and dual-I/O scheme. This approach not only reduces the chip area overhead but also enables reconfigurable GEMV/GEMM operations, boosting the performance. Simulations for the OPT 66B model show that our proposed accelerator achieves 2.9X, 3.5X, and 2.5X higher performance compared to NPU, DRAM-PIM, and heterogeneous designs (DRAM-PIM + NPU), respectively.

I. INTRODUCTION

Transformer-based large language models (LLMs) [1]–[3] have emerged as pivotal components in various language processing applications, ranging from chatbots to machine translations [4]–[6]. Moreover, these models are extending their influence into other domains such as vision applications and the integration of multiple domains [7]–[10]. LLMs demand substantial external memory due to their vast number of parameters, placing a strain on memory data bandwidth. Consequently, state-of-the-art deep learning platforms, including graphic processing units (GPUs) and neural processing units (NPUs), lean towards utilizing high bandwidth memory (HBM) as the primary memory source.

DRAM processing-in-memory (PIM)-based architectures [11]–[13] have been proposed to significantly boost data bandwidth, leveraging bank-level parallelism. However, DRAM-PIMs face challenges in achieving high performance for end-to-end LLM processing. The advent of multi-batching techniques [14]–[16] underscores a growing need to enhance LLM inference throughput by increasing the proportion of compute-intensive matrix-matrix multiplications (GEMM), which can lead to a degradation in the effectiveness of DRAM-PIM system optimized for memory-bound matrix-vector multiplications (GEMV). To overcome the challenge, heterogeneous systems which employ both NPU and DRAM-PIM together have been recently proposed [17], [18]. While the systems show performance improvement over NPU-only or DRAM-PIM-only system by selectively assigning GEMV and GEMM operations to DRAM-PIM and NPU, achieving equal load balancing between PIM and NPU is challenging, making it difficult to increase resource utilization. Furthermore, Tall-and-Skinny GEMM (TS-GEMM) appearing in multi-batch LLM inference shows both memory-intensive and compute-intensive traits, making both NPU and DRAM-PIM suffer.

To address these multifaceted challenges, we focus on the potential of Hybrid Bonding (HB) technology which provides high-density vertical interconnect between stacked silicon chips, enabling high performance and efficiency [19]. Recently, several accelerators based on 3D DRAM-to-Logic HB interconnect have been reported to demonstrate outstanding performance by exploiting the high compute density and high memory bandwidth which HB technology offers [20]–[22]. However, we observe that several design challenges still remain for efficient accelerator architecture based on HB, especially for

LLM applications. First, previous works on near-memory processors based on logic-to-DRAM HB technology [20]–[22] placed a DRAM controller for each DRAM bank to handle data at the bank level to transfer control signals via the bank level 3D HB interconnect. However, assigning a controller for each bank incurs significant area overhead [21]. Second, operating frequency of processing elements is limited to the HB-DRAM bank-level data rate which lowers the performance of the compute-bound GEMM in LLM inference.

To address these challenges, we propose a centralized controller based on a dual I/O scheme, which exploits both conventional DRAM I/O structure and bank-level I/O. The conventional structure I/O for external access in HB-DRAM does not utilize the benefit from bank-level parallelism of HB. However, we notice that its bus structure connects to multiple banks so that the control signals can be broadcast to the banks from the centralized memory controller. Therefore we make the centralized controller send the control signals to banks and the banks just *drop* the data to logic die. Replacing the distributed controller with the centralized controller saves significant area, enabling higher compute density. The proposed dual I/O enables reconfigurable GEMM/GEMV operations with variable operating frequency. Our design demonstrates higher frequency GEMM operations by transferring data at higher per-pin data rate using conventional DRAM I/O as well as lower frequency yet massively parallel GEMV operations using bank-level HB-interconnect.

Our main contributions can be summarized as follows:

- We present the dual I/O scheme and a centralized DRAM controller. The schemes provides significant area savings over previous designs that employed distributed DRAM controller. The dual I/O scheme enables reconfigurable GEMM/GEMV operations with variable operating frequency of processing elements: higher frequency GEMM operations as well as lower frequency yet massively parallel GEMV operations.
- Based on the proposed concepts, we present HB-NPU, a 3D DRAM-to-logic HB technology-based processing-near-memory (PNM) LLM inference accelerator architecture that can support both GEMM and GEMV operations efficiently.
- We demonstrate that HB-NPU exhibits 2.9X, 3.5X, and 2.5X higher performance than NPU, DRAM-PIM, and the heterogeneous system respectively, on OPT-66B using the custom simulator based on DRAMSim3 simulator [23].

II. BACKGROUND

A. Memory bottleneck in LLM inference

LLM [1]–[3] inference is divided into two phases: prefill and decoding. These phases exhibit significantly different operation intensities (OI). Prefill, characterized by parallel processing of multiple tokens, is dominated by compute-bound GEMM operations, whereas decoding, processing only one token at a time, is dominated by memory-bound GEMV operations. Batching [14]–[16] allows concatenation of tokens from multiple client requests that share the same weight matrix, enabling the conversion of GEMV operations

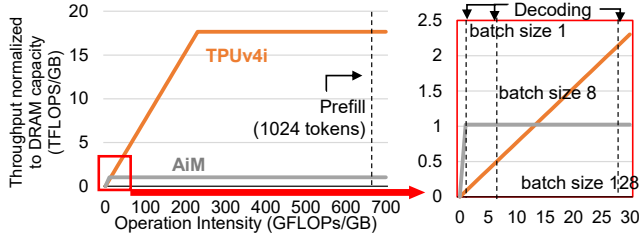


Fig. 1: Roofline model of representative NPU and DRAM-PIM; Google TPUv4i [24], and SK Hynix AiM [11]. Operation intensities for OPT-13B [3] inference with BF16 format and context length of 1024 are used. In the Y-axis, throughput with the same DRAM capacity is shown to compare TPUv4i and AiM which have different DRAM capacities.

in decoding phase to GEMM operations. However, the activation-activation multiplication in attention layer still need to be done with memory-bound GEMV operations even in the multi-batching scenario. Fig. 1 shows the roofline model of state-of-the-art accelerators for OPT-13B model [3]. While TPUv4i [24], a representative NPU, has abundant compute resources, AiM [11], a DRAM-PIM, has extremely high memory bandwidth thanks to its high bank-level parallelism. Given the same DRAM capacity, NPU outperforms the DRAM-PIM in prefill phase with high operation intensity because DRAM-PIM has a much lower compute density than NPU. On the other hand, decoding phase shows extremely low operation density, resulting in substantial memory bottlenecks in NPU. As a result, an efficient GEMV unit as well as an efficient GEMM unit is desired for an efficient LLM inference. With multi-batching, NPU can outperform DRAM-PIM even in decoding phase. Nevertheless, the performance is still much lower than its maximum throughput. This is because attention layers still depend on memory-bound GEMV operations, and it is hard to increase the batch size due to the strain on the memory capacity by memory parameters and KV cache, making input matrix shape tall and skinny (i.e. called Tall-and-Skinny GEMM or TS-GEMM). The size of KV cache increases with the batch size linearly so that the batch size is typically limited to less than 64 in real LLM inference servings [25]. Consequently, it is also desired to develop hardware optimized for TS-GEMM acceleration as well as reconfigurable GEMM/GEMV.

B. Hybrid Bonding (HB) Technology

Hybrid bonding (HB) has emerged as a promising alternative to traditional solder bumps, driven by the limitations in scaling solder bump pitch. In 3D System-in-Package (SiP) designs, where efficient chip-to-chip connections are essential, growing data transfer demands can lead to increased power consumption, causing communication inefficiencies. HB technology, with its exceptionally small sub-10 μ m pitch and low electrical loading, addresses these challenges by enabling more connections, higher bandwidth, and reduced power consumption for data transfer.

Fig. 2 illustrates the wafer-to-wafer HB stacking process in a face-to-face configuration, showcasing the cross-section of the stacked wafers. The top wafer is flipped to align face-down with the bottom wafer, allowing direct contact between HB pads on the top metal layer. This configuration significantly reduces connection thickness, leading to lower parasitic capacitance and resistance and achieving a connection quality similar to on-chip interconnects [19]. TSVs on the bottom die provide off-chip interfaces and power delivery.

III. PROPOSED CENTRALIZED CONTROLLER AND DUAL-I/O

A. Distributed I/O Channels in Previous HB DRAM

The I/O configuration of previous HB DRAM shows a distinct structure compared to conventional DRAM, as depicted in Fig. 3a.

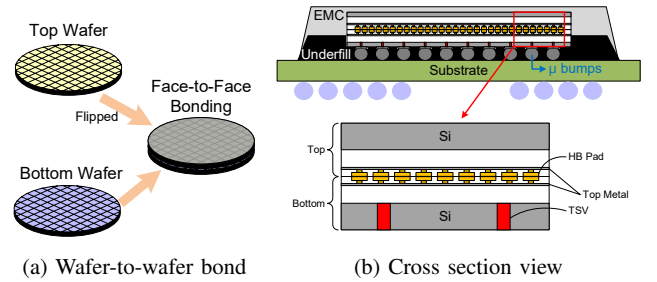


Fig. 2: Illustration of 3D-stacked chip using hybrid bonding

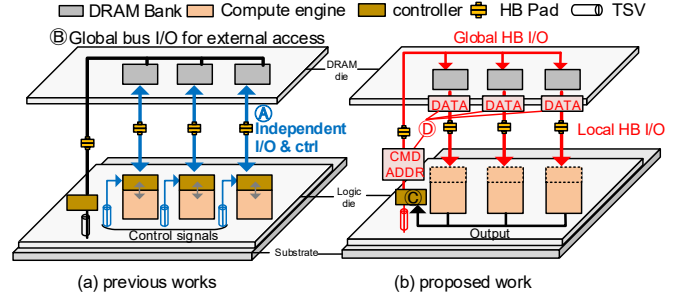


Fig. 3: (a) Distributed memory controllers in previous HB DRAM [20]–[22] and (b) the proposed centralized memory controller in our design. Note that extra TSVs for power delivery are omitted here.

HB DRAM supports independent I/O and controllers for each bank [20]–[22] enabling higher bandwidth through multiple connections between the DRAM die and logic die (A). This configuration provides significant advantages in data transfer rates over conventional DRAM systems. However, this distributed memory controller design incurs significant area overhead. Each bank requires dedicated components, including individual control paths, address decoder, timing controls, and Built-In-Self-Test (BIST) units, of which area consumes approximately 17% of a bank area, as reported in [21].

Additionally, the structure requires HB connections not only between memory banks and compute engines but also for off-chip communication via a conventional DRAM I/O global bus shared among multiple banks (B). Since the substrate is attached to the logic die, the off-chip datapath from the DRAM banks must pass through HB connections to interface with the TSVs on the backside of the logic die. We leverage this unused I/O with low data bandwidth in previous PNMs to enhance performance and efficiency.

B. Proposed Centralized controller and Dual-I/O

To overcome the limitations in the distributed memory controller-based designs, we propose a centralized controller and dual HB I/O between DRAM and NPU: Local bank-to-engine HB I/O and Global HB I/O (Fig. 3b). We eliminate local memory controllers for each compute engine, and instead use a centralized memory controller for global bus I/O to broadcast the control signals to banks (C). In our design, local HB I/O is used to read the data from multiple DRAM banks to corresponding logic in parallel, and global HB I/O is used to read and write the data with lower bandwidth. Data from multiple banks are read in parallel via local HB I/O once each DRAM bank receives command/address through global HB I/O, so each bank does not need a dedicated controller (D).

A previous work [21] claimed that distributed controllers were preferred in their vision AI accelerator design since centralized controllers require large area overhead to route and gather the distributed I/Os from different banks to the centralized controller. However, one of our key findings is that the centralized controller

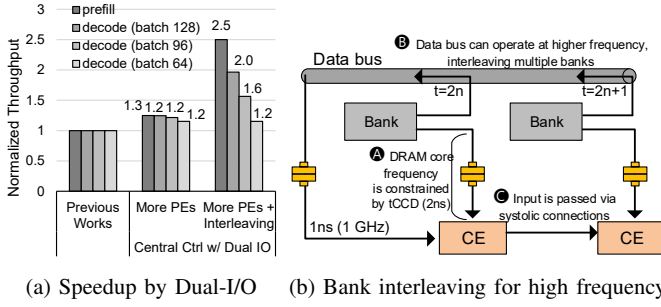


Fig. 4: (a) Analysis of speedup of the proposed design for OPT-13B with context length 1024 and various batch sizes. We can allocate more processing engines (PEs) by removing local memory controllers [21] and boost PE frequency by bank interleaving. (b) Bank interleaving using Global HB I/O.

is the better option for LLM accelerations. Firstly, the output traffic in matrix multiplication is smaller than the input traffic, so write operations require much lower bandwidth compared to read operations. Therefore, by using shared I/O instead of aggregating all distributed I/O, the number of I/O can be significantly reduced in write operation. Secondly, centralized control is more effective for DRAM access. Although the distributed controllers allow execution of different addresses for each bank in parallel, most operations in LLM, particularly Matrix Multiplication, exhibit regular memory access patterns, thus enabling to broadcast the same address to the banks simultaneously. For instance, in AiM [11], a PIM command is sent to all banks simultaneously with the same bank row and column addresses for executing matrix-vector multiplication.

By employing a centralized DRAM controller that handles both normal access and massively-parallel data access, the area overhead due to distributed controllers can be eliminated and the area can be used for extra compute resources. It enhances the compute density and throughput for compute-intensive layers in LLM. Replacing the distributed controller area with more computing resources results in approximately 1.2x-1.3x improvement in throughput for the OPT-13B model, as shown in Fig. 4a (labeled as *More PEs*).

Additionally, Global HB *data* I/O is useful for input data read in compute-bound operations since it can achieve a higher per-pin data rate than Local HB I/O for the same number of I/Os. By activating multiple banks in parallel and reading them in an interleaved manner, we can circumvent DRAM timing constraints and enhance per-pin data rate, thereby supporting higher logic frequencies. In our HB-NPU design example shown in Fig. 4b, processing engines (PEs) operate at 500MHz due to the 2ns of column-to-column delay (tCCD) timing constraint between each column-read command, when the input data directly come from DRAM banks via Local HB I/O (A). On the other hand, Global HB I/O can provide the higher 1GHz frequency by activating two banks in parallel and reading one of them alternately (B). To supply inputs from global I/O to all PEs, we organized all PEs connected via systolic connections to efficiently pass the inputs to neighboring PEs (C). Utilizing this bank interleaving technique, we use Global HB I/O to feed input data into the PE array during compute-bound GEMM operations, which do not require the high memory bandwidth provided by Local HB I/Os. Such variable frequency operations using the proposed dual I/O further improve the throughput as shown in Fig. 4a, particularly in prefill and larger batches with high operation intensities.

IV. HB-NPU: ARCHITECTURE AND DATAFLOW

Based on the aforementioned centralized controller and dual-I/O, we develop a prototype for an LLM inference accelerator, *HB-NPU*.

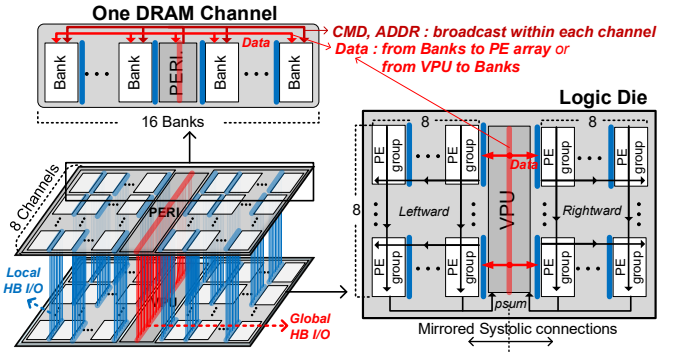


Fig. 5: Organization of the proposed HB-NPU.

For a fair comparison, we matched the compute-memory ratio of TPUv4i [24] in our design by constructing a single chip consisting of a 2GB DRAM die and a Logic die with a 128×128 systolic array and a Vector Processing Unit (VPU) with 128×8 lanes for BF16.

A. HB-NPU Organization

Fig. 5 shows the overall organization of the proposed HB-NPU architecture, which consists of a DRAM die and a logic die that are connected with HB interconnect. The DRAM bank array and the systolic array are tightly coupled so that a DRAM bank and the corresponding group of Multiply-Accumulate (MAC) processing engines (PEs) are placed at the same coordinate in each die and connected via Local HB I/O vertically. Typically, DRAM peripheral circuits for off-chip interface are located at the center of the DRAM die. Consequently, we also place VPU and peripheral units at the center of the Logic die and connect the logic units with DRAM via Global HB I/O. VPU handles the post-processing of output before writing it back to DRAM via Global HB I/O. The PE array is split into two 128×64 configurations to support the placement.

Memory Organization The DRAM die in HB-NPU consists of 8 channels with 16 banks per channel as shown in Fig. 5. The control signals and data coming into DRAM via Global HB I/O (red line) are broadcast across all banks within the channel but not shared across channels. The column data I/O from each bank is 256-bits that can serve 16 PEs (BF 16) at once. To feed the 128 PEs in a column simultaneously, vertical 8 banks must be always activated. To reduce the bank activation overhead, we align the channels with the PE array column. The DRAM timing constraint, tFAW (Four Activation Window) limits the number of banks *per channel* that can be activated within a specified time window which incurs significant latency overhead.

PE Array Organization As depicted in Fig. 5, following the 8 channels with 16 banks per channel configuration of the DRAM, the PE array in the Logic die is also structured into PE groups with an 8×16 configuration. The left and right sides of the VPU are mirrored, facilitating the connection of Global I/O to the PE interconnection and enabling data transferred into the PE array.

Unlike the DRAM-PIM [11] which can place only 16×1 PEs for each DRAM bank due to the area limitation, HB-NPU places 16×8 PEs as a *PE group* for each DRAM bank counterpart, thereby increasing compute resource significantly. Each DRAM bank supplies input and weight data to the PE group via the Local HB I/O. The final partial sum values are sent to VPU for accumulation and post-processing such as activation functions, normalization, softmax. The completed output data is sent back from VPU to the DRAM die via Global HB I/O and stored in banks.

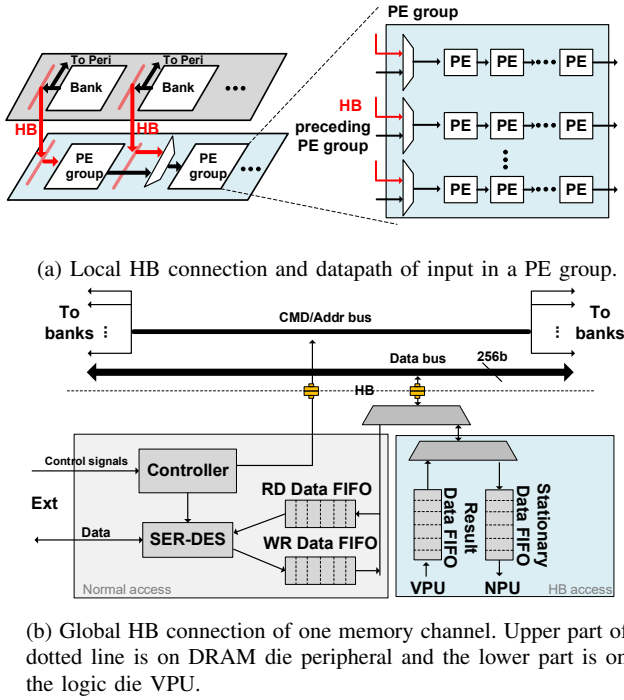


Fig. 6: Datapaths using Dual HB I/O connections.

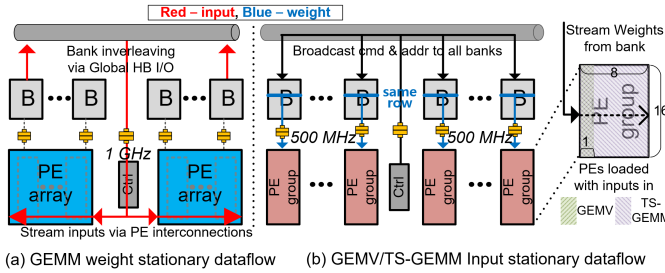


Fig. 7: Reconfigurable dataflow of HB-NPU. We use weight stationary flow with higher frequency for GEMM and input stationary with all-bank activation via Local HB for GEMV/TS-GEMM with lower frequency.

Datapaths using Dual HB I/O Fig. 6 shows the datapaths of Local HB I/O and Global HB I/O. The Local HB I/Os stream data from each DRAM bank to the PE array directly (Fig. 6a). Data coming from the bank either goes to the DRAM peripheral or moves through the Local HB I/O, where the consecutive columns of a bank row are streamed into the PE array sequentially. The first PE columns of each group of PEs can select the input data either from the preceding PE group or upper DRAM bank via the Local HB I/O. Fig. 6b shows the Global I/O datapath of one memory channel. Commands and addresses are sent to the banks by the centralized controller, and data, multiplexed between the external path and the NPU in the logic die, is communicated to the data bus in the DRAM die shared by the banks within each channel. Except for the external I/O interface including controller, Serializer-Deserializer (SER-DES), and data FIFOs, we have two extra FIFOs for result data from VPU and stationary data into NPU. The output data comes out from the systolic array and enters into Result Data FIFO and input data is supplied through the Stationary Data FIFO to load the PE registers. Therefore, we design Dual-IO support with minimal modification of Result Data FIFO and Stationary Data FIFO, exploiting existing DRAM I/Os.

B. Reconfigurable GEMM/GEMV dataflow of HB-NPU

Since each PE group in HB-NPU can receive input either vertically from DRAM or horizontally from the preceding PE group, we can selectively configure the dataflow for GEMM or GEMV kernels based on the given workloads. We adopt the weight-stationary dataflow for GEMM and input-stationary dataflow for GEMV considering the impact of DRAM timing constraints. Weight-stationary dataflow is efficient for GEMM operations since weights have a high chance of reuse. On the other hand, weight-stationary dataflow is inefficient in GEMV case since there is no reuse chance of weights when supplying inputs, necessitating frequent DRAM bank row switching to load the next weight tile. DRAM bank row switching causes substantial energy consumption and significant delay due to timing constraints. Therefore, it is more efficient to preload the input into PE registers and reuse it multiple times by streaming the weight matrix. Consequently, we opt for input-stationary dataflow for GEMV operations in our design.

GEMM Operations For GEMM, as shown in Fig. 7a, weight data is preloaded into PE registers, and input data is streamed into the PEs in a systolic fashion via Global HB I/O at 1 GHz frequency thanks to the bank interleaving technique. As described in Fig. 4b, Global HB I/O can boost memory bandwidth by accessing several banks in an interleaved manner, enabling 1 GHz frequency operation which is 2X higher frequency than the case when Local HB I/O is used.

GEMV Operations For GEMV, conversely, we preload inputs via Global HB I/O. Then, we activate all DRAM banks for memory-intensive weight streaming into PEs via local HB I/O's by broadcasting the command and address to all banks (Fig. 7b). While the operation frequency of PEs is restricted by the DRAM timing constraint by using Local HB I/O, massively parallel weight streaming significantly accelerates GEMV operations compared to conventional NPUs. As described in section III-B, matrix multiplication in LLM does not require a complex memory access pattern. Therefore, we can easily map LLM parameters on DRAM to access the same row of all DRAM banks simultaneously similar to conventional DRAM-PIM architectures. The large PE array in our HB-NPU is divided into 16 PE groups, and in each PE group, only the first PE column is involved in GEMV computation since the streamed weights can be computed by only one input vector.

Tall-and-Skinny GEMM Meanwhile, batch sizes in LLM inference are often limited in multi-batching scenarios due to the pressure on memory capacity by KV cache burden, making inputs become a tall and skinny matrix. On a conventional systolic array (e.g. 128×128), such tall and skinny GEMM (TS-GEMM) workloads lead to reduced PE utilization. The proposed HB-NPU has a unique feature to address both PE utilization and memory bandwidth issues by utilizing input stationary dataflow for TS-GEMM. We run TS-GEMM with high memory bandwidth by streaming weights into the PE groups via Local HB I/Os, while keeping inputs stationary in PE registers. As shown in Fig. 7b (right), larger number of PE columns in each PE group participate in the TS-GEMM computation than the GEMV dataflow in which only the first PE column in each PE group is utilized.

Data Mapping Each input and weight tile is divided into 8 memory channels and distributed to all PE rows via Global HB I/O. Weights are prearranged in DRAM prior to execution. For the activation-activation multiplication in attention layers, we evenly allocate different heads to each PE group, optimizing PE utilization. For arranging input data, since Global HB I/O can access any DRAM banks, VPU can send the completed output data of producer layers to the target

bank locations via Global HB I/O to make them ready for processing of the next consumer layers.

V. EVALUATION

A. Methodology

Baselines We assess our architecture against three types of representative LLM accelerators: NPU, DRAM-PIM, and heterogeneous system (NPU+PIM). All baselines and HB-NPU have the same 32GB capacity of DRAM on a board and the boards are interconnected by PCIe 4.0 16x for scaling-out to simulate large-size models. Detailed specifications are provided in Table I.

- **NPU:** NPU baseline features a $4 \times 128 \times 128$ systolic array with 8 GB HBM2 and vector processors referring to [24]. We simulated the NPU with ScaleSim cycle-level simulator [26].
- **DRAM-PIM:** DRAM-PIM baseline has a similar configuration to [11], that has 16 processing elements per bank and one global buffer per channel. We built a DRAM-PIM simulator based on the DRAMSim3 simulator [23].
- **Heterogeneous system (NPU+PIM):** Recently, heterogeneous systems combining NPUs and DRAM-PIMs [17], [18] have been proposed to accelerate both GEMM and GEMV operations in batched LLM decoding. Note that the heterogeneous system has 2x larger DRAM capacity than that of both the baseline and HB-NPU, with a half allocated to NPU and the other half to PIM. We assigned FC layers with GEMM operations to the NPUs and Attention layers with GEMV operations to the PIMs with NPU-PIM pipelining techniques that enhance system utilization.

Cycle-level simulation of HB-NPU We built a cycle-level simulator for HB-NPU based on the DRAMSim3 simulator¹ [23]. We added custom commands for HB-NPU operations and implemented a custom command generator for BLAS functions. We developed a custom Python program to generate a DRAM trace file from model information and memory specifications. Our program allows users to input various LLM workload information (e.g., number of heads, hidden dimensions, etc.), dataflow options, and DRAM specifications (DRAM capacity, device width, etc.). Then the program divides the operand data and allocates them to the physical address of the bank. We also consider chip-to-chip communication overhead for large-scale system. The generated trace file is read into the simulator and our custom command generator then derives DRAM commands from the provided transactions.

LLM Workloads We simulated the OPT model variations [3] across four different parameter sizes for multi-batch inference of LLM. We conducted experiments for batch sizes of 32, 64, and 128 using two token lengths, (1024, 128) and (128, 1024) for input and output tokens. Model parallelism is employed to distribute parameters and KV caches across multiple devices, with a mixture of pipeline parallelism (PP) [27] and tensor parallelism (TP) [28]. While high TP increases communication overhead, high PP divides a batch into several micro-batches, leading to PE under-utilization. Table II shows the LLM configuration with the model parallelism; Tensor parallelism (TP) and Pipeline parallelism (PP) we exploited.

B. Results

Decoding results Fig. 8 shows the simulation results for throughput of baselines and HB-NPU for multi-batch inference cases. While NPU throughput increases as the batch size increases, DRAM-PIM exhibits relatively low throughput because it is optimized to GEMV operations. The heterogeneous system, incorporating both NPU and

TABLE I: Hardware specification of baselines and HB-NPU.

NPU	Composition	4 128×128 PE arrays, 4 128×8 vector units 144 MB SRAM, 1 GHz
	Memory	2 4H HBM2 of 8 GB (1 GB/slice); off-chip bandwidth of 614 GB/s
	System configuration	4 chips/board: 32GB with 16 128×128 PEs Inter-chip bandwidth of 50 GB/s
PIM	Memory configuration	4H Stacked DRAM of 8 GB (2 GB/die) 8 channels/die, 16 banks/channel Column I/O 256 b, row size 2KB Core frequency 500 MHz
	Timing parameters	$t_{RCD} = 14ns$, $t_{RP} = 14ns$ $t_{CCD} = 2ns$, $t_{FAW} = 30ns$
	PEs	500 MHz, 16-lane MAC/bank
	System configuration	4 chips/board: 32 GB with 32K PEs 2 KB input buffer per channel
Ours	Memory die	2 GB Capacity, 8 channels, 16 banks/channel Column I/O 256b, row size 2KB Core frequency 500 MHz
	Timing parameters	$t_{RCD} = 14ns$, $t_{RP} = 14ns$ $t_{CCD} = 2ns$, $t_{FAW} = 30ns$
	Logic die	8x16 PGs, 16x8 PEs per PG 128x8 vector unit, 1MB output accumulator 1 GHz(GEMM)/500 MHz(GEMV, TS-GEMM)
	System configuration	16 chips/board: 32GB with 16 128×128 PEs Inter-chip bandwidth of 50 GB/s

TABLE II: OPT and model parallelism configuration. *Heterogeneous system has the twice larger memory capacity. +HB-NPU has 4x higher TPs due to lower chip capacity.

Model	#L	#H	d_{model}	(TP ⁺ , PP)	DRAM Capacity*
2.7B	32	32	2560	(8,1)	64 GB
6.7B	32	32	4096	(8,2)	128 GB
13B	40	40	5120	(16,2)	256 GB
66B	64	72	9216	(32,4)	1024 GB

PIM modules, demonstrates higher performance than NPU-only or DRAM-PIM-only baselines. However, TS-GEMM workloads with low batch sizes cause both NPU and PIM modules to suffer compared to HB-NPU. The HB-NPU, with its support for a reconfigurable systolic array and high bandwidth, demonstrates the highest throughput compared to the baselines. Overall, our HB-NPU shows 3.0x, 4.1x, and 2.0x performance improvement on average over NPU, DRAM-PIM, and the heterogeneous system (DRAM-PIM + NPU), respectively. The higher performance of HB-NPU is particularly noticeable for larger model with lower batch size, which is expected to be a main workload in inference servings due to the increasing demands for larger AI model and longer context [33], [34].

Fig. 9 shows the breakdown of runtime among various inference engines. Fig. 9a represents the case in which computation of attention layer is dominant with long input sequence, while Fig. 9b represents the case when the FC layer is dominant with large model dimension. In Fig. 9a, the NPU is suffering from inefficiency in GEMV computations, because of the large size of KV cache with long input sequence. As batch size increases, both NPU and PIM exhibit significantly larger runtime increases compared to the other two systems, due to their lack of ability to accelerate GEMV or GEMM, respectively. In Fig. 9b, the runtimes for both the NPU and the heterogeneous system (which includes an NPU) decrease as the batch size increases, leading to higher PE utilization. In contrast, DRAM-PIM's computation time for the FC layer increases linearly with batch size, making it significantly slower than the other systems. In both scenarios, HB-NPU exhibits the best performance, effectively computing both FC and attention layers. While heterogeneous system also effectively computes both layers, they feature dedicated processing units for GEMM and GEMV so that the mismatch in the ratio of GEMM

¹HB-NPU simulator is available at <https://github.com/shh8228/HB-NPU-simulator.git>

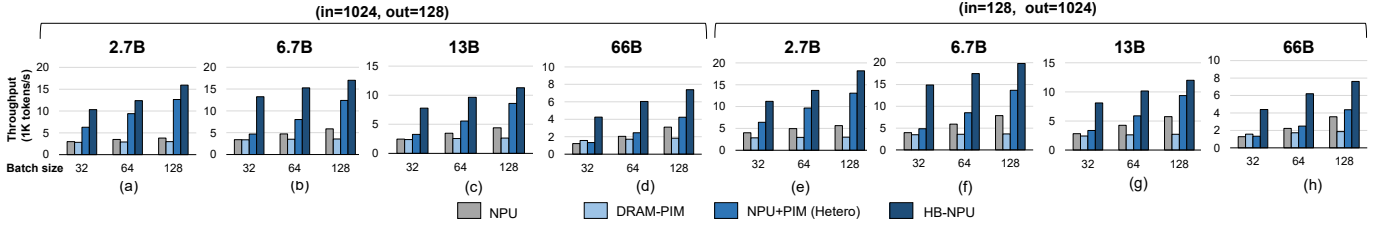


Fig. 8: Throughput comparison for decoding in OPT models with various batch sizes and token sizes.

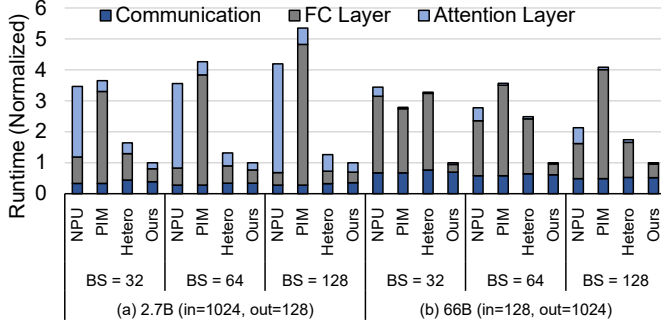
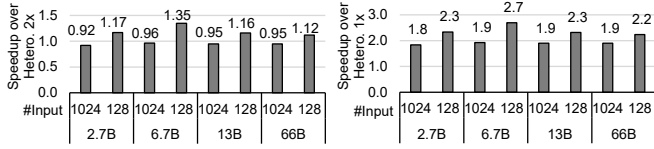


Fig. 9: Runtime breakdown for decoding, normalized to ours (HB-NPU).



(a) Heterogeneous system with doubled DRAM capacity. (b) Heterogeneous system with the same DRAM capacity as HB-NPU.

Fig. 10: Prefill speedup of HB-NPU compared to heterogeneous system with (a) doubled DRAM capacity and (b) same DRAM capacity as HB-NPU.

to GEMV operations leads to lower PE utilization. In contrast, HB-NPU utilizes reconfigurable processing units, leading to superior performance across various scenarios.

Prefill results Fig. 10a shows the speedup of HB-NPU in prefill, compared to heterogeneous system. HB-NPU shows higher performance than heterogeneous system with 128 input tokens due to the lower operation intensity. With the larger token size 1024, heterogeneous system is slightly faster than HB-NPU because heterogeneous system utilizes larger system resources than HB-NPU. However, considering the difference in the memory capacity and number of PEs, HB-NPU achieves higher efficiency than heterogeneous system.

Implication of DRAM/PE resources in Heterogeneous System In the experimental setting for Fig. 8 and 10a, we assumed that the heterogeneous system has twice larger DRAM size and more PEs than other baselines and our HB-NPU as the heterogeneous system consists of an NPU with dedicated DRAM and a DRAM-PIM similar to [18]. Therefore, for a fair comparison, it is worth comparing HB-NPU with the heterogeneous system that has the same DRAM capacity. Fig. 10b and Fig. 11 show that the performance gap between our HB-NPU and the heterogeneous system increases as the DRAM capacity and the number of PEs in the heterogeneous system are reduced.

C. Area and Energy Analysis

Area overhead of VPU To assess whether the VPU unit can be implemented in the area budget of a conventional DRAM peripheral area [29], we first synthesized the VPU of 128x16 ALUs with

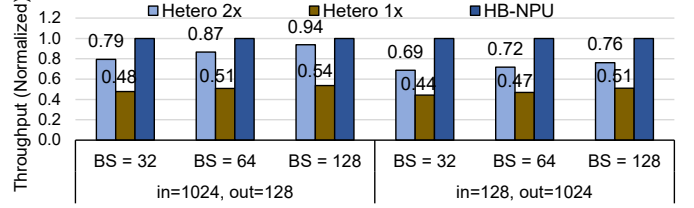


Fig. 11: Throughput comparison of HB-NPU (1024 GB DRAM) with the heterogeneous system with 2048 GB DRAM (Hetero 2x) and 1024 GB DRAM (Hetero 1x) for OPT-66B. Throughput values are normalized to that of HB-NPU.

accumulation buffer following the TPU series setup [30], with a 28nm technology. Then, we scaled the area to 14nm using the equation from [32]. We also included the area of DRAM Physical layer (PHY) based on the HBM3 specification. We observed that the VPU and DRAM PHY consume 83.5% of the target area budget, implying that no extra area is required.

Energy analysis Our HB-NPU consumes much lower energy than other baselines, due to its energy-efficient dataflow using Dual-I/O. HB-NPU consumes lower energy than NPU due to the short HB interconnect. HB-NPU also consumes lower energy than DRAM-PIM thanks to its more energy-efficient GEMM dataflow than that of DRAM-PIM. Overall, simulation results using DRAMsim3 [23] show that HB-NPU consumes about 19x, 9x, 3x lower energy than DRAM-PIM, NPU, and heterogeneous system, respectively, on the OPT-2.7B model.

VI. CONCLUSION

We introduce HB-NPU, a versatile PNM architecture for LLM inference designed to accelerate both GEMM and GEMV operations leveraging 3D DRAM-to-Logic HB technology. HB-NPU alleviates memory bottlenecks and enhances compute resource density through its centralized controller and dual HB I/O schemes. This reconfigurability for GEMM/GEMV operations is significant, as most existing AI accelerators are optimized for either GEMM or GEMV. Moreover, HB-NPU's support for tall-and-skinny GEMM enables higher performance in LLM inference involving limited batch size. The combination of these features positions HB-NPU as a promising solution for efficient, high-performance LLM inference processing.

VII. ACKNOWLEDGEMENT

This work was supported in part by SK Hynix and Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (RS-2023-00208606, NeuroHub+: Scheduler and Simulator for General In-Memory Neural Network Accelerators, IITP-2023-RS-2023-00256081: artificial intelligence semiconductor support program to nurture the best talents), BK21 FOUR program at Seoul National University, and ISRC at Seoul National University. The EDA tool was supported by the IC Design Education Center(IDECE). (Corresponding Author: Jae-Joon Kim).

REFERENCES

- [1] T. Brown, *et al.*, "Language models are few-shot learners," *NeurIPS*, 2020.
- [2] H. Touvron, *et al.*, "Llama: Open and efficient foundation language models," *arXiv preprint*, 2023.
- [3] S. Zhang, *et al.*, "Opt: Open pre-trained transformer language models," *arXiv preprint*, 2022.
- [4] J. Wei, *et al.*, "Leveraging large language models to power chatbots for collecting user self-reported data," *HCI*, 2024.
- [5] S. Pujar, *et al.*, "Invited: Automated Code generation for Information Technology Tasks in YAML through Large Language Models," *DAC*, 2023.
- [6] B. Zhang, *et al.*, "Prompting large language model for machine translation: a case study," *ICML*, 2023.
- [7] W. Wang, *et al.*, "Visionllm: Large language model is also an open-ended decoder for vision-centric tasks," *NeurIPS*, 2023.
- [8] D. Gufran, *et al.*, "VITAL: Vision Transformer Neural Networks for Accurate Smartphone Heterogeneity Resilient Indoor Localization," *DAC*, 2023.
- [9] Q. Ye, *et al.*, "Mplug-octopus: The versatile assistant empowered by a modularized end-to-end multimodal llm," *ACMMM*, 2023.
- [10] C. Yang, *et al.*, "Late Breaking Results: Fast Fair Medical Applications? Hybrid Vision Models Achieve the Fairness on the Edge," *DAC*, 2023.
- [11] S. Lee, *et al.*, "A 1ynm 1.25v 8gb, 16gb/s/pin gddr6-based accelerator-in-memory supporting 1tflops mac operation and various activation functions for deep-learning applications," *ISSCC*, 2022.
- [12] Y.-C. Kwon, *et al.*, "25.4 a 20nm 6gb function-in-memory dram, based on hbm2 with a 1.2tflops programmable computing unit using bank-level parallelism, for machine learning applications," *ISSCC*, 2021.
- [13] J. Park, *et al.*, "NTT-PIM: Row-Centric Architecture and Mapping for Efficient Number-Theoretic Transform on PIM," *DAC*, 2023.
- [14] G.-I. Yu, *et al.*, "Orca: A distributed serving system for Transformer-Based generative models," *OSDI*, 2022.
- [15] W. Kwon, *et al.*, "Efficient memory management for large language model serving with pagedattention," *SOSP*, 2023.
- [16] R. Pope, *et al.*, "Efficiently scaling transformer inference," *MLSys*, 2023.
- [17] G. Heo, *et al.*, "Neupims: Npu-pim heterogeneous acceleration for batched llm inferencing," *ASPLOS*, 2024.
- [18] J. Park, *et al.*, "Attacc! unleashing the power of pim for batched transformerbased generative model inference," *ASPLOS*, 2024.
- [19] B. Ayoub, *et al.*, "Impact of process variations on the capacitance and electrical resistance down to 1.44 μm hybrid bonding interconnects," *EPTC*, 2020.
- [20] D. Niu, *et al.*, "184qps/w 64mb/mm23d logic-to-dram hybrid bonding with process-near-memory engine for recommendation system," *ISSCC*, 2022.
- [21] Z. Yue, *et al.*, "Exploiting similarity opportunity of emerging ai models on 3d hybrid bonding architecture," *ISCA*, 2024.
- [22] B. Fujun, *et al.*, "A stacked embedded dram array for lpddr4/4x using hybrid bonding 3d integration with 34gb/s/1gb 0.88pj/b logic-to-memory interface," *IEDM*, 2020.
- [23] "Dramsim3," <https://github.com/umdmemsys/DRAMsim3>.
- [24] N. P. Jouppi, *et al.*, "Ten lessons from three generations shaped google's tpuv4i : Industrial product," *ISCA*, 2021.
- [25] P. Patel, *et al.*, "Splitwise: Efficient Generative LLM Inference Using Phase Splitting," *ISCA*, 2024.
- [26] A. Samajdar, *et al.*, "Scale-sim: Systolic cnn accelerator simulator," *arXiv preprint*, 2019.
- [27] Y. Huang, *et al.*, "Gpipe: Efficient training of giant neural networks using pipeline parallelism," *NeurIPS*, 2019.
- [28] D. Narayanan, *et al.*, "Efficient large-scale language model training on gpu clusters using megatron-lm," *SC*, 2021.
- [29] M.-J. Park, *et al.*, "A 192-gb 12-high 896-gb/s hbm3 dram with a tsv auto-calibration scheme and machine-learning-based layout optimization," *JSSC*, vol. 58, no. 1, pp. 256–269, 2023.
- [30] T. Norrie, *et al.*, "The design process for google's training chips: Tpuv2 and tpuv3," *IEEE Micro*, vol. 41, no. 2, pp. 56–63, 2021.
- [31] N. P. Jouppi, *et al.*, "In-datacenter performance analysis of a tensor processing unit," *ISCA*, 2017.
- [32] A. Stillmaker, *et al.*, "Scaling equations for the accurate prediction of cmos device performance from 180nm to 7nm," *Integration*, vol. 58, pp. 74–81, 2017.
- [33] A. Chowdhery, *et al.*, "Palm: Scaling language modeling with pathways," *Journal of Machine Learning Research*, 2023.
- [34] Gemini Team, *et al.*, "Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context," *arXiv preprint*, 2024.